МОСКОВСКИЙ АВИАЦИОННЫЙ ИНСТИТУТ
(НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ)

Институт №8 «Компьютерные науки»

Лабораторная работа №1
по курсу «Технология параллельного программирования и программирование графических процессоров»

Освоение программного обеспечения для работы с технологией CUDA. Примитивные операции над векторами.

Выполнил: Курносов М.Ю.
Группа: М8О-406Б-23
Преподаватель: А.Ю. Морозов

Москва, 2026

Условие

Цель работы: Ознакомление и установка программного обеспечения для работы с программно-аппаратной архитектурой параллельных вычислений (CUDA).
Реализация одной из примитивных операций над векторами.
В качестве вещественного типа данных необходимо использовать тип данных double. Все результаты выводить с относительной точностью 10.
Ограничение: $-10 \; n < 2$.

Вариант 3. Поэлементное умножение векторов.

Входные данные. На первой строке задано число n -- размер векторов. В следующих 2-х строках, записано по n вещественных чисел -- элементы векторов.

Выходные данные. Необходимо вывести n чисел -- результат поэлементного умножения исходных векторов.

Программное и аппаратное обеспечение

Аппаратное обеспечение (Hardware)
Графический процессор (GPU): NVIDIA Tesla T4

- Compute Capability: 7.5 (соответствует архитектуре Turing).
- Графическая память (Global Memory): 15360 МБ (15 ГБ) GDDR6.
- Разделяемая память (Shared Memory): 48 КБ на блок (может быть расширена до 96 КБ за счет L1-кэша).
- Константная память (Constant Memory): 64 КБ (общая для всех потоков).
- Количество регистров на блок: 65536 (максимум).
- Максимальное количество нитей (threads) в блоке: 1024.
- Максимальное количество блоков в сетке (grid): $231-1231-1$ по каждому измерению.
- Количество мультипроцессоров (SM): 40 (стандартная конфигурация для T4).

Процессор (CPU): Обычно это виртуализированный Intel Xeon @ 2.20GHz или AMD EPYC (зависит от дата-центра Google). Для отчета можно указать: *Intel Xeon Processor (Skylake/Cascade Lake), 2 vCPU*.
Оперативная память (RAM): ~13 ГБ (доступно пользователю в среде Colab).
Жесткий диск (Storage): Виртуальный диск, объем зависит от загруженных файлов и временных данных сессии (обычно доступно несколько десятков ГБ).
Программное обеспечение (Software)
Операционная система: Ubuntu 24.04.1 LTS (64-bit).
Среда разработки (IDE): Jupyter Notebook (интерфейс Google Colaboratory).
Компилятор CUDA: nvcc версии 12.8 (NVIDIA Cuda compilation tools, release 12.8).
Компилятор C++: g++ версии 13.3.0 (или актуальной для данного образа Ubuntu).
Драйвер NVIDIA: Версия 580.82.07.

Метод решения

Общий алгоритм Задача решается путем параллельного поэлементного сложения двух векторов: Ci=Ai+Bi*Ci=Ai+Bi*, где *i* — индекс элемента. Тип данных для вычислений — double для обеспечения требуемой относительной точности (10−1010−10).
Архитектура программы Программа построена по модели гетерогенных вычислений (CPU + GPU):

1. Хостовая часть (CPU): отвечает за чтение данных из stdin, выделение памяти, управление передачей данных и вывод результата в stdout.
2. Устройство (GPU): выполняет арифметические операции над элементами векторов в параллельных потоках.

Стратегия распараллеливания (Grid-Stride Loop) Поскольку по требованиям количество вычислительных потоков должно быть фиксированным и не зависеть от размера вектора N*N*, используется паттерн Grid-Stride Loop (цикл с шагом сетки).

- При запуске ядра создается фиксированная сетка блоков (например, 10 блоков по 1024 потока).
- Каждый поток вычисляет свой уникальный глобальный индекс: i = blockIdx.x * blockDim.x + threadIdx.x.
- Внутри ядра поток в цикле обрабатывает элементы массива, увеличивая индекс на шаг, равный общему количеству потоков (stride = gridDim.x * blockDim.x), пока не достигнет конца массива N*N*. Этот подход гарантирует, что программа корректно обработает вектор любого размера (в пределах ограничения N<230*N*<230), даже если N*N* многократно превышает количество доступных потоков.

Описание программы

Проект состоит из двух файлов:
lab1.cu — исходный код программы, содержащий как хостовую часть (функция main), так и устройство (CUDA-ядро).
lab1.c — Файл содержит последовательную (однопоточную) реализацию алгоритма сложения векторов на центральном процессоре. Используется как эталон для проверки корректности результатов, полученных на GPU, а также для замера времени выполнения и расчета ускорения (speedup).
Makefile — скрипт сборки, использующий компилятор nvcc с флагами для архитектуры sm_75 и строгой проверки ошибок (-Werror).
Основные типы данных
double — используется для хранения элементов векторов и проведения вычислений. Выбор обусловлен требованием обеспечения относительной точности
10−10.
long long (64-битное целое) — используется для переменной n
n (размер вектора) и индексов внутри ядра. Это необходимо для безопасной адресации памяти, так как n может достигать значений близких к 2^30, что превышает безопасные пределы для 32-битного int при умножении на размер типа.

cudaError_t — перечисляемый тип для хранения кодов возврата функций CUDA при проверке на ошибки.
Описание реализованных ядер и функций
Ядро vectorAdd
Сигнатура: __global__ void vectorAdd(const double* A, const double* B, double* C, long long n)
Описание: Выполняет поэлементное сложение двух векторов.
Архитектура: Реализован паттерн Grid-Stride Loop (цикл с шагом сетки).
Каждый поток вычисляет свой уникальный стартовый индекс: i = blockIdx.x * blockDim.x + threadIdx.x.
Вычисляется общий шаг (stride) для всех потоков: stride = blockDim.x * gridDim.x.
В цикле while(i < n) поток выполняет операцию C[i] = A[i] + B[i] и сдвигается на stride.
Преимущество: Такой подход позволяет обрабатывать векторы произвольного размера (в пределах ограничения n<2^30n<2^30) при строго фиксированном количестве запущенных потоков, что полностью удовлетворяет требованиям методички.
Хостовая функция main
Описание: Управляет жизненным циклом программы.
Логика:
Считывает n и значения векторов из stdin.
Выделяет память на CPU (malloc) и на GPU (cudaMalloc).
Копирует входные вектора на видеокарту (cudaMemcpy с флагом cudaMemcpyHostToDevice).
Запускает ядро vectorAdd с фиксированной конфигурацией (например, 10 блоков по 1024 потока).
Копирует результат с GPU на CPU (cudaMemcpyDeviceToHost).
Выводит результирующий вектор в stdout с заданной точностью.
Освобождает выделенную память (free и cudaFree).
Макрос обработки ошибок ERROR_PRINT
Описание: Инкапсулирует вызов cudaGetLastError(). Если функция CUDA завершилась сбоем, макрос выводит строку ERROR: <описание ошибки> в stderr и принудительно завершает программу с кодом возврата 0, предотвращая аварийное падение (Segmentation Fault).

Результаты

	Lab1.cu			Lab1.c		
Длинна вектора	5000000	5000	5	5000000	5000	5
<<< 1, 32 >>>	0m3.491s	0m0.386s	0m0.393s	0m3.283s	0m0.011s	0m0.009s
<<< 32, 32 >>>	0m3.662s	0m0.386s	0m0.303s			
<<< 32, 64 >>>	0m3.651s	0m0.393s	0m0.278s			
<<< 32, 1024 >>>	0m3.812s	0m0.384s	0m0.286s			
<<< 1024, 1024 >>>	0m3.512s	0m0.393s	0m0.397s			

Выводы

В ходе лабораторной работы реализовано сложение векторов на GPU с использованием CUDA.

1. Применен паттерн **grid-stride loop**, позволяющий использовать фиксированное число потоков для обработки массивов любого размера.
2. Использованы типы double (для точности 10−1010−10) и long long (для безопасной адресации больших массивов).
3. Реализована обработка ошибок: программа не аварийно не падает, а выводит сообщение в stderr и возвращает код 0.
4. Сравнение с CPU-версией показало, что на малых данных основное время тратится на копирование памяти, а вычислительная мощность GPU раскрывается только на предельных объемах.

Цель работы достигнута: получен опыт создания гетерогенных приложений и управления ресурсами видеокарты.